PaXini

API 参考



PaXini Tech

GMH15



CONTENTS

PaXini

1. 控制类 DexH15Control 01

2. 相机控制类 DexH15Camera 37

3. 运动学解析类 DexH15Kinematic 40

1. 控制类 DexH15Control

负责灵巧手的串口通信、电机控制、传感器读取等核心操作。

1.1 connectModbusAuto

- **功能概述**

  自动连接当前串口下的第一个灵巧手设备。适用于单设备场景，系统将自动完成设备发现与连接流程。

- **函数原型**

```
1  bool connectModbusAuto(const std::string& hand_port_num, int16_t& device_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
hand_port_num	const std::string&	输入	灵巧手连接的串口编号（如："/dev/ttyUSB0"）
device_address	int16_t&	输出	成功连接后返回的第一个设备地址

- **返回值**

  true: 连接成功
  false: 连接失败

- **特别说明**

  程序执行过程中发生错误，则会抛出异常。

1.2 openModbusDevice

- **功能概述**

  打开指定串口设备，建立 Modbus 通信连接。

- **函数原型**

```
1  bool openModbusDevice(const std::string& hand_port_num, int32_t baud_rate);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
hand_port_num	const std::string&	输入	灵巧手连接的串口编号
baud_rate	int32_t	输入	串口设备通讯的波特率（4000000, 921600, 460800, 230400, 115200, 57600）

- **返回值**

  true: 打开成功
  false: 打开失败

1.3 scanModbusDevices

- **功能概述**

  扫描已打开串口下所有可通讯的设备地址。

- **函数原型**

```
1  std::vector<int16_t> scanModbusDevices(const std::string& hand_port_num);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
hand_port_num	const std::string&	输入	灵巧手连接的串口编号

- **返回值**

  返回已打开串口下所有可通讯设备地址的集合（std::vector<int16_t>）。

1.4 initModbusDevice

- **功能概述**

  初始化灵巧手 Modbus 设备配置。

- **函数原型**

```
1  int initModbusDevice(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

  1: 初始化成功
  0: 初始化失败

1.5 disconnectModbus

- **功能概述**

关闭当前 Modbus 串口连接，释放串口资源。

- **函数原型**

```
bool disconnectModbus();
```

- **参数说明**

无

- **返回值**

true: 断开成功
false: 断开失败

1.6 isModbusDeviceConnected

- **功能概述**

检查指定从站地址的灵巧手 Modbus 连接状态。

- **函数原型**

```
bool isModbusDeviceConnected(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

true: 设备已连接
false: 设备未连接

1.7 setModbusBaudrate

- **功能概述**

设置 Modbus 从站的通信波特率。

- **函数原型**

```
bool setModbusBaudrate(int16_t slave_address, int32_t baud_rate);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
baud_rate	int32_t	输入	目标波特率（4000000, 921600, 460800, 230400, 115200, 57600）

- **返回值**

true: 设置成功
false: 设置失败

- **特别说明**

- 波特率的设置必须在使用串口前完成，设置完成后需重启设备方可生效。
- 同一串口上挂载的所有设备波特率必须保持一致。

1.8 getModbusBaudrate

- **功能概述**

获取当前 Modbus 从站的通信波特率。

- **函数原型**

```
int32_t getModbusBaudrate(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回从站设备当前的波特率值（int32_t）。

1.9 setModbusDeviceAddress

- **功能概述**

修改 Modbus 从站的设备地址。

- **函数原型**

```
bool setModbusDeviceAddress(int16_t slave_address, int16_t new_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	当前从站地址（0x01 ~ 0xF7）
new_address	int16_t	输入	新的从站地址（0x01 ~ 0xF7）

- **返回值**

true: 设置成功
false: 设置失败

- **特别说明**

从站地址的设置必须在使用前完成，设置完成后需重启设备方可生效。

1.10 openUsbDevice

- **功能概述**

打开 USB 串口设备并建立连接，自动获取从站地址。

- **函数原型**

```
bool openUsbDevice(const std::string& hand_port_num, int16_t& device_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
hand_port_num	const std::string&	输入	USB 设备编号
device_address	int16_t&	输出	成功连接后返回的从站地址

- **返回值**

true: 打开成功
false: 打开失败

1.11 disconnectUsb

- **功能概述**

关闭 USB 设备连接，释放相关资源。

- **函数原型**

```
bool disconnectUsb();
```

- **参数说明**

无

- **返回值**

true: 操作执行完成

1.12 setMotorControlMode

- **功能概述**

设置电机的控制模式。默认模式为**位置模式**，可根据需要切换为电流模式或速度模式。

- **函数原型**

```
bool setMotorControlMode(int16_t slave_address, const ControlMode mode);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
mode	const ControlMode	输入	电机控制模式枚举值

- **ControlMode 枚举值说明**

枚举值	数值	说明
CURRENT_CONTROL_MODE	1	电流环控制模式
VELOCITY_CONTROL_MODE	2	速度环控制模式
POSITION_CONTROL_MODE	3	位置环控制模式（默认）

- **返回值**

true: 设置成功
false: 设置失败

1.13 getHandType

- **功能概述**

查询灵巧手的左右手类型。

- **函数原型**

```
1  HandType getHandType(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 HandType 枚举值，表示灵巧手的左右手类型。

枚举值	数值	说明
NONE_HAND	0	无效灵巧手类型
LEFT_HAND	1	左手
RIGHT_HAND	2	右手

1.14 setMotorMaxCurrent

- **功能概述**

设置所有电机的最大电流，用于限制握力峰值。

- **函数原型**

```
1  bool setMotorMaxCurrent(int16_t slave_address, const Dex15MotorCurrent& max_current);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
max_current	const Dex15MotorCurrent&	输入	各电机最大电流配置，取值范围 [0, 1000]

- **返回值**

true: 设置成功
false: 设置失败

1.15 getMotorMaxCurrentConfig

- **功能概述**

获取当前从站设备所有电机的最大电流配置。

- **函数原型**

```
1  Dex15MotorCurrent getMotorMaxCurrentConfig(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 Dex15MotorCurrent 结构体，包含所有电机的最大电流配置。

1.16 setMotorMaxVelocity

- **功能概述**

设置所有电机的最大速度。

- **函数原型**

```
1  bool setMotorMaxVelocity(int16_t slave_address, const Dex15MotorSpeed& max_speed);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
max_speed	const Dex15MotorSpeed&	输入	各电机最大速度配置，取值范围 [0, 1000]

- **返回值**

true: 设置成功
false: 设置失败

1.17 getMotorMaxVelocityConfig

- **功能概述**

获取当前从站设备所有电机的最大速度配置。

- **函数原型**

```
1  Dex15MotorSpeed getMotorMaxVelocityConfig(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 Dex15MotorSpeed 结构体，包含所有电机的最大速度配置。

1.18 enableMotor

- **功能概述**

使能（上电激活）指定从站的所有电机。

- **函数原型**

```
bool enableMotor(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

true: 使能成功
false: 使能失败

1.19 disableMotor

- **功能概述**

下使能（断电）指定从站的所有电机。

- **函数原型**

```
bool disableMotor(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

true: 下使能成功
false: 下使能失败

1.20 isMotorEnabled

- **功能概述**

查询指定从站电机的当前使能状态。

- **函数原型**

```
bool isMotorEnabled(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

true: 电机已使能
false: 电机未使能

1.21 setJointPositionsAngle（全关节）

- **功能概述**

控制全部 11 个手指关节的目标角度（**归一化角度**）。

- **函数原型**

```
int setJointPositionsAngle(int16_t slave_address, const std::vector<double>& joint_angles);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
joint_angles	const std::vector<double>&	输入	11 个可调关节的归一化角度值集合

- **joint_angles 元素说明**

顺序依次为: 食指（2 个）→ 中指（2 个）→ 无名指（2 个）→ 小拇指（2 个）→ 大拇指（3 个）。

索引	关节标识	对应手指	取值范围	说明
0	index_joint1	食指	[0, 1000]	近端关节，弯曲伸展
1	index_joint2	食指	[0, 1000]	末端关节，弯曲伸展
2	middle_joint1	中指	[0, 1000]	近端关节，弯曲伸展
3	middle_joint2	中指	[0, 1000]	末端关节，弯曲伸展
4	ring_joint1	无名指	[0, 1000]	近端关节，弯曲伸展
5	ring_joint2	无名指	[0, 1000]	末端关节，弯曲伸展
6	little_joint1	小拇指	[0, 1000]	近端关节，弯曲伸展
7	little_joint2	小拇指	[0, 1000]	末端关节，弯曲伸展
8	thumb_joint1	大拇指	[-500, 500]	掌指双轴关节，侧位摆动
9	thumb_joint2	大拇指	[0, 1000]	近端关节，弯曲伸展
10	thumb_joint3	大拇指	[0, 1000]	末端关节，弯曲伸展

- **返回值**

1: 设置成功
0: 设置失败

1.22 setJointPositionsAngle（指定手指）

• 功能概述

设置指定单根手指的关节归一化角度。

• 函数原型

```
int setJointPositionsAngle(int16_t slave_address, FingerType finger_id, const std::vector<double>& joint_angles);
```

• 参数说明

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
finger_id	FingerType	输入	目标手指的枚举值
joint_angles	const std::vector<double>&	输入	指定手指的归一化角度值集合

• FingerType 枚举值定义

枚举值	数值	说明
INVALID_FINGER	0	无效值
INDEX_FINGER	1	食指
MIDDLE_FINGER	2	中指
RING_FINGER	3	无名指
PINKY_FINGER	4	小拇指
THUMB_FINGER	5	大拇指

• 返回值

1: 设置成功
0: 设置失败

1.23 calculateRealAngle（控制类）

• 功能概述

根据归一化角度值，计算对应的真实物理角度（单位：度）。

• 函数原型

```
int calculateRealAngle(int16_t slave_address,
                       const std::vector<double>& normalize_angle,
                       std::vector<double>& real_angle);
```

• 参数说明

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
normalize_angle	const std::vector<double>&	输入	11 个可调关节的归一化角度值集合
real_angle	std::vector<double>&	输出	计算得到的 11 个关节真实角度值集合

• normalize_angle 元素说明

顺序依次为：食指（2 个）→ 中指（2 个）→ 无名指（2 个）→ 小拇指（2 个）→ 大拇指（3 个）。



索引	关节标识	对应手指	取值范围	说明
0	index_joint1	食指	[0, 1000]	近端关节，弯曲伸展
1	index_joint2	食指	[0, 1000]	末端关节，弯曲伸展
2	middle_joint1	中指	[0, 1000]	近端关节，弯曲伸展
3	middle_joint2	中指	[0, 1000]	末端关节，弯曲伸展
4	ring_joint1	无名指	[0, 1000]	近端关节，弯曲伸展
5	ring_joint2	无名指	[0, 1000]	末端关节，弯曲伸展
6	little_joint1	小拇指	[0, 1000]	近端关节，弯曲伸展
7	little_joint2	小拇指	[0, 1000]	末端关节，弯曲伸展
8	thumb_joint1	大拇指	[-500, 500]	掌指双轴关节，侧位摆动
9	thumb_joint2	大拇指	[0, 1000]	近端关节，弯曲伸展
10	thumb_joint3	大拇指	[0, 1000]	末端关节，弯曲伸展

• 返回值

1: 计算成功
0: 计算失败

1.24 calculateNormalizeAngle（控制类）

- **功能概述**

根据真实物理角度值，计算对应的归一化角度。

- **函数原型**

```
int calculateNormalizeAngle(int16_t slave_address,
                            const std::vector<double>& real_angle,
                            std::vector<double>& normalize_angle);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
real_angle	const std::vector<double>&	输入	11 个可调关节的真实角度值集合（单位：度）
normalize_angle	std::vector<double>&	输出	计算得到的 11 个可调关节归一化角度值集合

- **real_angle 元素说明**

顺序依次为：食指（2 个）→ 中指（2 个）→ 无名指（2 个）→ 小拇指（2 个）→ 大拇指（3 个）。

索引	关节标识	对应手指	取值范围	说明
0	index_joint1	食指	[0, 80]	近端关节，弯曲伸展
1	index_joint2	食指	[0, 80]	末端关节，弯曲伸展
2	middle_joint1	中指	[0, 80]	近端关节，弯曲伸展
3	middle_joint2	中指	[0, 80]	末端关节，弯曲伸展
4	ring_joint1	无名指	[0, 80]	近端关节，弯曲伸展
5	ring_joint2	无名指	[0, 80]	末端关节，弯曲伸展
6	little_joint1	小拇指	[0, 80]	近端关节，弯曲伸展
7	little_joint2	小拇指	[0, 80]	末端关节，弯曲伸展
8	thumb_joint1	大拇指	[0, 109.6]	掌指双轴关节，侧位摆动
9	thumb_joint2	大拇指	[0, 87]	近端关节，弯曲伸展
10	thumb_joint3	大拇指	[0, 120.1]	末端关节，弯曲伸展

- **返回值**

1：计算成功
0：计算失败

1.25 setMotorTargetCurrent（全部电机）

- **功能概述**

在**电流模式**下，设置所有电机的目标电流。目标电流值越大，手的握力越强。

- **函数原型**

```
int setMotorTargetCurrent(int16_t slave_address, int16_t target_current);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
target_current	int16_t	输入	全部电机的统一目标电流，取值范围 [-1000, 1000]

- **返回值**

1：设置成功
0：设置失败

1.26 setMotorTargetCurrent（指定电机）

- **功能概述**

在**电流模式**下，设置指定单个电机的目标电流。

- **函数原型**

```
int setMotorTargetCurrent(int16_t slave_address,
                          FingerMotor motor_id,
                          int16_t target_current);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
motor_id	FingerMotor	输入	目标电机的枚举索引值
target_current	int16_t	输入	目标电流值，取值范围 [-1000, 1000]

- **FingerMotor 枚举值定义**

枚举值	数值	说明
INVALID_MOTOR	0	无效值
INDEX_MOTOR	1	食指电机
MIDDLE_MOTOR	2	中指电机
RING_MOTOR	3	无名指电机
PINKY_MOTOR	4	小拇指电机
THUMB_MOTOR_1	5	大拇指侧摆电机
THUMB_MOTOR_2	6	大拇指电机 2
THUMB_MOTOR_3	7	大拇指电机 3

- **返回值**

1：设置成功
0：设置失败

1.27 getMotorActualCurrent

- **功能概述**

获取当前所有电机的实际电流值。

- **函数原型**

```
int getMotorActualCurrent(int16_t slave_address, Dex15MotorCurrent& actual_current);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
actual_current	Dex15MotorCurrent&	输出	包含所有电机的实际电流, 范围 [-1000, 1000]

- **返回值**

1: 获取成功
0: 获取失败

1.28 setMotorTargetVelocity（全部电机）

- **功能概述**

在**速度模式**下, 统一设置所有电机的目标速度, 用于控制手的整体运动速度。

- **函数原型**

```
int setMotorTargetVelocity(int16_t slave_address, int16_t target_speed);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
target_speed	int16_t	输入	全部 7 个电机的统一目标速度, 取值范围 [-1000, 1000]

- **返回值**

1: 设置成功
0: 设置失败

1.29 setMotorTargetVelocity（指定电机）

- **功能概述**

在**速度控制模式**下, 设置指定单个电机的目标速度。

- **函数原型**

```
int setMotorTargetVelocity(int16_t slave_address, FingerMotor motor_id, int16_t target_speed);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
motor_id	FingerMotor	输入	目标电机的枚举索引值
target_speed	int16_t	输入	目标速度值, 取值范围 [-1000, 1000]

- **FingerMotor 枚举值定义**

枚举值	数值	说明
INVALID_MOTOR	0	无效值
INDEX_MOTOR	1	食指电机
MIDDLE_MOTOR	2	中指电机
RING_MOTOR	3	无名指电机
PINKY_MOTOR	4	小拇指电机
THUMB_MOTOR_1	5	大拇指侧摆电机
THUMB_MOTOR_2	6	大拇指电机 2
THUMB_MOTOR_3	7	大拇指电机 3

- **返回值**

1: 设置成功
0: 设置失败

1.30 getMotorActualVelocity

- **功能概述**

获取当前所有电机的实际运行速度。

- **函数原型**

```
int getMotorActualVelocity(int16_t slave_address, Dex15MotorSpeed& actual_speed);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
actual_speed	Dex15MotorSpeed&	输出	包含所有电机的实际速度, 范围 [-1000, 1000]

- **返回值**

1: 获取成功
0: 获取失败

1.31 calibrateSensor

- **功能概述**

触觉传感器标定。

- **函数原型**

```
bool calibrateSensor(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

true: 标定成功
false: 标定失败

1.32 getJointPositionsAngle（全关节）

- **功能概述**

获取手指关节的当前归一化角度。当 is_all_joint 为 false 时获取 11 个可调关节角度; 为 true 时额外返回 4 个末端指骨（DIP）关节, 共获取 15 个关节角度。

- **函数原型**

```
int getJointPositionsAngle(int16_t slave_address, std::vector<double>& joint_angles, bool is_all_joint = false);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
joint_angles	std::vector<double>&	输出	关节归一化角度值集合, 元素个数由 is_all_joint 决定
is_all_joint	bool	输入	是否获取全部关节角度, 默认 false（获取 11 个可调关节）

is_all_joint = false 时, joint_angles 元素说明（11 个, 顺序: 食指→中指→无名指→小拇指各 2 个, 大拇指 3 个）

索引	关节标识	对应手指	取值范围	说明
0	index_joint1	食指	[0, 1000]	近端关节, 弯曲伸展
1	index_joint2	食指	[0, 1000]	末端关节, 弯曲伸展
2	middle_joint1	中指	[0, 1000]	近端关节, 弯曲伸展
3	middle_joint2	中指	[0, 1000]	末端关节, 弯曲伸展
4	ring_joint1	无名指	[0, 1000]	近端关节, 弯曲伸展
5	ring_joint2	无名指	[0, 1000]	末端关节, 弯曲伸展
6	little_joint1	小拇指	[0, 1000]	近端关节, 弯曲伸展
7	little_joint2	小拇指	[0, 1000]	末端关节, 弯曲伸展
8	thumb_joint1	大拇指	[-500, 500]	掌指双轴关节, 侧位摆动
9	thumb_joint2	大拇指	[0, 1000]	近端关节, 弯曲伸展
10	thumb_joint3	大拇指	[0, 1000]	末端关节, 弯曲伸展

is_all_joint = true 时, joint_angles 元素说明（15 个, 四指各新增 1 个末端指骨 DIP 关节）

索引	关节标识	对应手指	取值范围	说明
0	index_joint1	食指	[0, 1000]	近端关节, 弯曲伸展
1	index_joint2	食指	[0, 1000]	中端关节, 弯曲伸展
2	index_joint3	食指	[0, 1000]	末端指骨关节, 弯曲伸展
3	middle_joint1	中指	[0, 1000]	近端关节, 弯曲伸展
4	middle_joint2	中指	[0, 1000]	中端关节, 弯曲伸展
5	middle_joint3	中指	[0, 1000]	末端指骨关节, 弯曲伸展
6	ring_joint1	无名指	[0, 1000]	近端关节, 弯曲伸展
7	ring_joint2	无名指	[0, 1000]	中端关节, 弯曲伸展
8	ring_joint3	无名指	[0, 1000]	末端指骨关节, 弯曲伸展
9	little_joint1	小拇指	[0, 1000]	近端关节, 弯曲伸展
10	little_joint2	小拇指	[0, 1000]	中端关节, 弯曲伸展
11	little_joint3	小拇指	[0, 1000]	末端指骨关节, 弯曲伸展
12	thumb_joint1	大拇指	[-500, 500]	掌指双轴关节, 侧位摆动
13	thumb_joint2	大拇指	[0, 1000]	近端关节, 弯曲伸展
14	thumb_joint3	大拇指	[0, 1000]	末端关节, 弯曲伸展

- **返回值**

1: 获取成功
0: 获取失败

1.33 getJointPositionsAngle（指定手指）

- **功能概述**

获取指定单根手指的当前关节角度。当 is_all_joint 为 false 时该手指返回 2 个关节（大拇指为 3 个）；为 true 时该手指额外返回 1 个末端指骨（DIP）关节。

- **函数原型**

```
int getJointPositionsAngle(int16_t slave_address, FingerType finger_id, std::vector<double>& joint_angles,
                           bool is_all_joint = false);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
finger_id	FingerType	输入	目标手指的枚举索引值
joint_angles	std::vector<double>&	输出	指定手指的关节角度值集合，元素个数由 is_all_joint 决定
is_all_joint	bool	输入	是否获取该手指全部关节角度，默认 false

- **返回值**

1：获取成功
0：获取失败

1.34 getMotorControlMode

- **功能概述**

获取当前电机的控制模式。

- **函数原型**

```
ControlMode getMotorControlMode(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 ControlMode 枚举值：

参数名	类型	输入 / 输出
NONE	0	无效值（仅在获取失败时出现）
CURRENT_CONTROL_MODE	1	电流环控制模式
VELOCITY_CONTROL_MODE	2	速度环控制模式
POSITION_CONTROL_MODE	3	位置环控制模式

1.35 getFingerTactile [deprecated]

- **功能概述**

【已弃用】此函数已不再维护，请改用 getFingerResultantForce。获取全部手指触觉传感器的**合力**信息。手共有 5 个传感器，每个传感器返回一个三轴合力。

- **函数原型**

```
1    std::vector<ForcePoint> getFingerTactile(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回包含 5 个 ForcePoint 的列表，每个 ForcePoint 包含：

字段	类型	说明
x	float	传感器 X 轴力分量，单位 0.1N
y	float	传感器 Y 轴力分量，单位 0.1N
z	float	传感器 Z 轴力分量，单位 0.1N

1.36 getFingerResultantForce

- **功能概述**

获取全部手指触觉传感器的合力信息。手共有 5 个传感器，每个传感器返回一个三轴合力。

- **函数原型**

```
1    std::vector<ForcePoint> getFingerResultantForce(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回包含 5 个 ForcePoint 的列表，每个 ForcePoint 包含：

字段	类型	说明
x	float	传感器 X 轴力分量，单位 0.1N
y	float	传感器 Y 轴力分量，单位 0.1N
z	float	传感器 Z 轴力分量，单位 0.1N

1.37 getFingerTactileDetail [deprecated]

- **功能概述**

【已弃用】此函数已不再维护，请改用 getFingerDistributedForce。获取全部手指触觉传感器的**分布力**详细信息。手共有 5 个传感器，总计 337 个分布力点。

- **函数原型**

```
1    std::vector<ForcePoint> getFingerTactileDetail(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回包含 327 个 ForcePoint 的列表。分布规则如下：

参数名	类型
食指	50
中指	50
无名指	50
小拇指	50
大拇指	127

每个 ForcePoint 包含 x、y、z 三轴分量，单位 0.1N。

1.38 getFingerDistributedForce

- **功能概述**

获取全部手指触觉传感器的分布力详细信息。手共有 5 个传感器，总计 327 个分布力点。

- **函数原型**

```
1    std::vector<ForcePoint> getFingerDistributedForce(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回包含 327 个 ForcePoint 的列表。分布规则如下：

参数名	类型
食指	50
中指	50
无名指	50
小拇指	50
大拇指	127

每个 ForcePoint 包含 x、y、z 三轴分量，单位 0.1N。

1.39 getTemperature

- **功能概述**

获取全部手指传感器的温度数据。手共有 5 个传感器，返回 5 个温度读数。

- **函数原型**

```
std::vector<FingerTemp> getTemperature(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回包含 5 个 FingerTemp 的列表，每个 FingerTemp 包含：

字段	类型	输入 / 输出
finger_name	std::string	手指名称
sensor_ID	int	传感器 ID
temperature	float	传感器温度值（1℃）

1.40 getFingerTactileData

- **功能概述**

一次性获取**指定单根手指**的合力、分布力以及温度数据。

- **函数原型**

```
void getFingerTactileData(int16_t slave_address,
                          FingerType finger_id,
                          ForcePoint& finger_tac,
                          std::vector<ForcePoint>& finger_tac_detail,
                          FingerTemp& temperature);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
finger_id	FingerType	输入	目标手指的枚举类型值
finger_tac	ForcePoint&	输出	该手指的触觉传感器合力信息
finger_tac_detail	std::vector<ForcePoint>&	输出	该手指的触觉传感器分布力信息
temperature	FingerTemp&	输出	该手指的传感器温度数据

- **FingerType 枚举值定义**

枚举值	数值	说明
INVALID_FINGER	0	无效值
INDEX_FINGER	1	食指
MIDDLE_FINGER	2	中指
RING_FINGER	3	无名指
PINKY_FINGER	4	小拇指
THUMB_FINGER	5	大拇指

- **返回值**

无返回值（void）。

1.41 getAllTactileData

- **功能概述**

一次性获取**所有手指**的合力、分布力以及温度数据。

- **函数原型**

```
void getAllTactileData(int16_t slave_address,
                       std::vector<ForcePoint>& finger_tac,
                       std::vector<ForcePoint>& finger_tac_detail,
                       std::vector<FingerTemp>& temperature);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
finger_tac	std::vector<ForcePoint>&	输出	全部手指的触觉传感器合力信息
finger_tac_detail	std::vector<ForcePoint>&	输出	全部手指的触觉传感器分布力信息
temperature	std::vector<FingerTemp>&	输出	全部手指的传感器温度数据
temperature	FingerTemp&	输出	该手指的传感器温度数据

- **返回值**

无返回值（void）。

1.42 checkHandStatus

- **功能概述**

检查灵巧手的整体运行状态。

- **函数原型**

```
int checkHandStatus(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

参数名	类型
1	正常运行
2	容错运行状态（出现了错误，但不影响核心接口使用）
3	不可运行状态

1.43 getFaultCode

- **功能概述**

获取当前电机的故障码，用于诊断电机故障原因。

- **函数原型**

```
1  std::vector<FingerMotorsError> getFaultCode(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 std::vector<FingerMotorsError> 故障码列表，包含各电机的当前故障信息。

错误名	错误码	错误描述	解释说明	解决办法
OVER_VOLT	0	手指电机过压	驱动供电电压高于 15V，持续 2 秒	电压恢复后，执行清除错误指令即可
UNDER_VOLT	1	手指电机欠压	驱动供电电压低于 10V，持续 2 秒	电压恢复后，执行清除错误指令即可
OVER_CURRENTU	2	手指电机 U 相过流	U 相电流过大	电流恢复正常后，执行清除错误指令即可
OVER_CURRENTY	3	手指电机 V 相过流	V 相电流过大	电流恢复正常后，执行清除错误指令即可
OVER_CURRENTW	4	手指电机 W 相过流	W 相电流过大	电流恢复正常后，执行清除错误指令即可
OVER_TEMPERATURE	5	手指电机过温	电机温度过高	等待手指电机温度恢复到正常范围后，执行清除错误指令即可
ADC_CALIB	6	手指电机硬件自检错误	AD 静态偏差 +10%	1、建议检查驱动附件的磁场环境，排除干扰后，断电重启即可 2、进行 1 检查后，还是无法无法消除，建议更换驱动板
FOC_ERR	7	手指电机 FOC 计算错误	无	无
OPD_IQ	8	手指电机 IQ 过流	电机 IQ 电流过大	IQ 电流过流消除后，执行清错执行即可
OPD_U	9	手指电机 U 相缺相	上电初始化时候进行的检测	建议检查电机 U 相线是否有断开，恢复后，执行清错执行即可
OPD_V	10	手指电机 V 相缺相	上电初始化时候进行的检测	建议检查电机 V 相线是否有断开，恢复后，执行清错执行即可
OPD_W	11	手指电机 W 相缺相	上电初始化时候进行的检测	建议检查电机 W 相线是否有断开，恢复后，执行清错执行即可
POS_REF_OVER	12	手指电机位置超上限	无	无
POS_REF_UNDER	13	手指电机位置超下限	无	无
ABZ_ERR	14	手指电机编码器异常	磁编码一圈丢失计数大于 0.25%	电机端编码器丢数，需要排查编码器，断电重启后，观察现象，一直报错则需返厂
ABZ_Z_ERR	15	手指电机编码器 Z 相异常	初始化时候寻不到 Z 相	电机端编码器 Z 相错误，需要排查编码器，断电重启后，观察现象，一直报错则需返厂
ABZ_CALIB_ERR	16	编码器校准错误	编码器线没接好，或者未校准	需要排查编码器或者接线
SPEED_REF_OVER	17	给定速度超限	无	无
CURRENT_REF_OVER	18	给定电流超限	无	无
JOINT_ENC_DISCONNECT	19	磁编断联	无	无
MAGN2_NEGA_WARN	27	磁编出现较小负值	磁编值小于 -180 大于 -360	无
MAGN2_NEGA_ERR	28	磁编出现较大负值	磁编值小于 -360	无
NORM_POS_NEGA_WARN	29	归一化电机行程出现较小负数	电机 6 行程值小于 -400 大于 -800 或其他电机行程小于 -220 大于 -440	无
NORM_POS_NEGA_ERR	30	归一化电机行程出现较大负数	电机 6 行程值小于 -800 或者其他电机行程小于 -400	无
DISCONNECT	31	手指电机未连接	手指驱动和控制板的通讯断开	连接后，系统会自动清错并恢复

1.44 clearFaultCode

- **功能概述**

清除当前电机的故障码。

- **函数原型**

```
1  bool clearFaultCode(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

true: 清除成功
false: 清除失败

- **特别说明**

并非所有故障码都能被清除，详情请参见 SDK 配套说明文档。

1.45getFirmwareVersion

- **功能概述**

获取设备的固件版本信息。

- **函数原型**

```
1  const FirmwareVersion getFirmwareVersion(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 FirmwareVersion 结构体，包含设备的固件版本详细信息。

1.46 getSDKVersion

- **功能概述**

获取当前 SDK 的版本号。

- **函数原型**

```
const std::string getSDKVersion();
```

- **参数说明**

无

- **返回值**

返回 SDK 版本号字符串（std::string）。

1.47 initMotorPosition

- **功能概述**

初始化所有电机位置，使灵巧手回到初始零位状态。

- **函数原型**

```
int initMotorPosition(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

1: 初始化成功
0: 初始化失败

- **特别说明**

建议在设备**上电后立即调用**，执行完成最大耗时约 **5 秒**。

1.48 setMotorTargetPosition（全部电机）

- **功能概述**

通过电机位置参数结构体，统一设置所有电机的目标位置。

- **函数原型**

```
int setMotorTargetPosition(int16_t slave_address,
                           const Dex15MotorPosition& target_position);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
target_position	const Dex15MotorPosition&	输入	所有电机的目标位置值

- **返回值**

1: 设置成功
0: 设置失败

1.49 setMotorTargetPosition（指定电机）

- **功能概述**

设置指定单个电机的目标位置。

- **函数原型**

```
int setMotorTargetPosition(int16_t slave_address,
                           FingerMotor motor_id,
                           int16_t target_position);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
motor_id	FingerMotor	输入	目标电机的枚举索引值
target_position	int16_t	输入	目标电机位置值 [0, 10000]

- **FingerMotor 枚举值定义**

枚举值	数值	说明
INVALID_MOTOR	0	无效值
INDEX_MOTOR	1	食指电机
MIDDLE_MOTOR	2	中指电机
RING_MOTOR	3	无名指电机
PINKY_MOTOR	4	小拇指电机
THUMB_MOTOR_1	5	大拇指侧摆电机
THUMB_MOTOR_2	6	大拇指电机 2
THUMB_MOTOR_3	7	大拇指电机 3

- **返回值**

1: 设置成功
0: 设置失败

1.50 getMotorPosition

- **功能概述**

获取当前所有电机的位置信息。

- **函数原型**

```
const Dex15MotorPosition getMotorPosition(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 Dex15MotorPosition 结构体, 包含所有电机的当前位置信息。

1.51 getPressPositionForce

- **功能概述**

获取当前所有手指按压力的位置力点集合, 单位为 0.1N。

- **函数原型**

```
std::vector<PressForcePoint> getPressPositionForce(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 std::vector<PressForcePoint>, 包含所有手指的按压力位置点数据, 单位 0.1N。

1.52 getDeviceSN

- **功能概述**

获取设备的 **SN** 序列号。

- **函数原型**

```
std::string getDeviceSN(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回设备 **SN** 码字符串。

1.53 getHardwareVersion

- **功能概述**

获取设备的硬件版本信息。

- **函数原型**

```
HardwareVersion getHardwareVersion(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 HardwareVersion 结构体, 包含设备硬件版本详细信息。

1.54 getModbusConfig

- **功能概述**

获取当前从站设备的 Modbus 通信配置信息。

- **函数原型**

```
ModbusConfig getModbusConfig(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 ModbusConfig 结构体, 包含当前 Modbus 通信配置。

1.55 getDeviceInfo

- **功能概述**

通过单次调用，统一获取从站设备的 SN 码、硬件版本、固件版本及 Modbus 配置等全部设备信息。

- **函数原型**

```
bool getDeviceInfo(int16_t slave_address,
                   std::string& sn,
                   HardwareVersion& hardware_version,
                   FirmwareVersion& firmware_version,
                   ModbusConfig& modbus_config);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
sn	std::string&	输出	设备 SN 码
hardware_version	HardwareVersion&	输出	设备硬件版本信息
firmware_version	FirmwareVersion&	输出	设备固件版本信息
modbus_config	ModbusConfig&	输出	设备 Modbus 通信配置信息

- **返回值**

true: 获取成功
false: 获取失败

1.56 getJointMagneticEncoder

- **功能概述**

获取当前从站设备所有关节的磁编码器原始数据。

- **函数原型**

```
std::vector<int16_t> getJointMagneticEncoder(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回包含所有关节磁编码器数值的列表（std::vector<int16_t>）。

1.57 getActionPosition

- **功能概述**

加载并读取内置或自定义动作序列对应的电机位置信息。

- **函数原型**

```
int getActionPosition(int16_t slave_address,
                      int16_t index,
                      Dex15MotorPosition& action_position,
                      bool is_customized = true);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
index	int16_t	输入	动作序列编号 （is_customized=true 时范围 [1, 20]； is_customized=tfalse 时范围 [1, 10]）
action_position	Dex15MotorPosition&	输出	获取到的电机位置数据
is_customized	bool	输入	true: 自定义动作； false: 官方内置动作 （默认 true）

- **返回值**

1: 获取成功
0: 获取失败

1.58 runAction

- **功能概述**

执行指定编号的内置或自定义动作。

- **函数原型**

```
int runAction(int16_t slave_address, int16_t index, bool is_customized = true);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
index	int16_t	输入	动作序列编号 （is_customized=true 时范围 [1, 20]； is_customized=false 时范围 [1, 10]）
is_customized	bool	输入	true: 自定义动作； false: 官方内置动作 （默认 true）

- **返回值**

1: 执行成功
0: 执行失败

1.59 saveCustomizedAction（指定动作帧）

- **功能概述**

将指定的电机位置数据保存为用户自定义动作。

- **函数原型**

```
int saveCustomizedAction(int16_t slave_address,
                         int16_t index,
                         const Dex15MotorPosition& action);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
index	int16_t	输入	自定义动作序列编号，范围 [1, 20]
action	const Dex15MotorPosition&	输入	要保存的电机位置数据

- **返回值**

1：保存成功
0：保存失败

1.60 saveCustomizedAction（当前位置）

- **功能概述**

将灵巧手**当前实时位置**直接保存为指定编号的自定义动作。

- **函数原型**

```
int saveCustomizedAction(int16_t slave_address, int16_t index);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
index	int16_t	输入	自定义动作序列编号，范围 [1, 20]

- **返回值**

1：保存成功
0：保存失败

1.61 saveCustomizedAction（关节角度）

- **功能概述**

根据指定的关节归一化角度集合，计算电机位置后保存为自定义动作。

- **函数原型**

```
int saveCustomizedAction(int16_t slave_address,
                         int16_t index,
                         std::vector<double> joint_angle_list);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
index	int16_t	输入	自定义动作序列编号，范围 [1, 20]
joint_angle_list	std::vector<double>	输入	要保存的关节归一化角度集合（11 个可调关节）

- **返回值**

1：保存成功
0：保存失败

1.62 setCommonForceCtlParam

- **功能概述**

配置普通力控的初始位置和控制参数。配置完成后，通过 computeCommonForceTargetCommand 计算每次运动的目标指令。

- **函数原型**

```
int setCommonForceCtlParam(int16_t slave_address,
                           const std::vector<double>& joint_angles,
                           const ForceCtlParam& param);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
joint_angles	const std::vector<double>&	输入	进行普通力控时的初始灵巧手位置，需传入 11 个可调关节的归一化角度值
param	const ForceCtlParam&	输入	力控参数配置，不设置时使用默认参数，详细说明如下： · impedance_factor：阻抗因子，数值等级越大力控速度越快。单个关节的值范围为 [1, 100] · reference_force_factor：参考力系数，数值等级越大对应的力越大。单个关节的值范围为 [1, 10] · enable_flag：除大拇指侧摆外的其他 6 个主动关节的力控效果使能标记，默认开启所有关节。关节顺序依次为：食指远端关节、中指远端关节、无名指远端关节、小拇指远端关节、大拇指关节 2、大拇指关节 3 注意：成员属性 impedance_factor 和 reference_force_factor 中的关节顺序依次为：食指近端口关节、食指远端关节、中指近端、中指远端、环指近端、环指远端、小拇指近端、小拇指远端、大拇指关节 1、大拇指关节 2、大拇指关节 3

- **返回值**

1: 配置成功
0: 配置失败

- **特别说明**

- 普通力控与切向力力控的参数配置会**相互覆盖**，两者不可同时设置使用。被覆盖后需重新设置参数，才能进行目标位置的计算。
- 灵巧手的当前位置与需要进行力控的初始位置 joint_angles 不同时，需要视情况休眠等待灵巧手到达初始位置。

1.63 computeCommonForceTargetCommand

- **功能概述**

在普通力控模式下，根据当前力传感器数据计算下一次运动的目标关节角度指令。

- **函数原型**

```
int computeCommonForceTargetCommand(int16_t slave_address, std::vector<double>& command);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
command	std::vector<double>&	输出	下次运动的归一化角度指令，包含 11 个可调关节数据

- **返回值**

1: 计算成功
0: 计算失败

1.64 setTangentialForceCtlParam

- **功能概述**

配置**切向力力控**的初始位置和控制参数。配置完成后，通过 computeTangentialForceTargetCommand 计算每次运动的目标指令。

- **函数原型**

```
int setTangentialForceCtlParam(int16_t slave_address,
                               const std::vector<double>& joint_angles,
                               const TanForceCtlParam& param);
```


- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
joint_angles	const std::vector<double>&	输入	进行切向力力控时的初始灵巧手位置，需传入 11 个可调关节的归一化角度值
param	const TanForceCtlParam&	输入	力控参数配置，不设置时使用默认参数，详细说明如下： · tangential_force_gain: 增益值，等级越大抓的越紧，默认增益等级值为 10，范围为 [1, 100] · impedance_factor: 阻抗因子，数值等级越大力控速度越快。需传入 11 个关节数据，单个关节的值范围为 [1, 100] · reference_force_factor: 参考力系数，数值等级越大对应的力越大。需传入 11 个关节数据，单个关节的值范围为 [1, 10] · enable_flag: 除大拇指侧摆外的其他 6 个主动关节的力控效果使能标记，默认开启所有关节。关节顺序依次为：食指远端关节、中指远端关节、无名指远端关节、小拇指远端关节、大拇指关节 2、大拇指关节 3 注意：成员属性 impedance_factor 和 reference_force_factor 中的关节顺序依次为：食指近端口关节、食指远端关节、中指近端、中指远端、环指近端、环指远端、小拇指近端、小拇指远端、大拇指关节 1、大拇指关节 2、大拇指关节 3

- **返回值**

1: 配置成功
0: 配置失败

- **特别说明**

- 普通力控与切向力力控的参数配置会**相互覆盖**，两者不可同时设置使用。被覆盖后需重新设置参数，才能进行目标位置的计算。
- 灵巧手的当前位置与需要进行力控的初始位置 joint_angles 不同时，需要视情况休眠等待灵巧手到达初始位置。

1.65 computeTangentialForceTargetCommand

- **功能概述**

在切向力力控模式下，根据当前力传感器数据计算下一次运动的目标关节角度指令。

- **函数原型**

```
int computeTangentialForceTargetCommand(int16_t slave_address,
                                        std::vector<double>& command);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
command	std::vector<double>&	输出	下次运动的归一化角度指令，包含 11 个可调关节数据

- **返回值**

1: 计算成功
0: 计算失败

1.66 upgradeFirmware

- **功能概述**

升级传感器固件版本。开始升级前请重新给灵巧手上电。

- **函数原型**

```
1  UpgradeErrorCode upgradeSensor(int16_t slave_address, const std::vector<UpgradeFileAndPart>& info);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
info	const std::vector<UpgradeFileAndPart>&	输入	传感器固件与升级部位列表; 升级文件需要和程序放在同一目录下

- **UpgradeFileAndPart 结构说明**

字段	类型	说明
file_path	int16_t	固件文件路径
upgrade_part	std::set<int16_t>	需要升级的传感器部位编号集合

- **返回值**

返回 UpgradeErrorCode 枚举值, 常见取值:

枚举值	数值	说明
UPGRADE_CHECK_OK	1001	升级检查正常
UPGRADE_FAIL	1002	升级失败
UPGRADE_SUCCESS	1003	升级成功
FILE_FORMAT_ERROR	2001	文件格式异常
FILE_SIZE_ERROR	2002	文件大小异常
FILE_MATCH_ERROR	2003	文件匹配异常
FILE_NAME_ERROR	2004	文件名异常
FILE_COUNT_ERROR	2005	文件数量异常
FILE_EXIT_ERROR	2006	文件不存在
FILE_VERSION_ERROR	2007	文件版本错误
FILE_HAND_TYPE_ERROR	2008	文件对应的灵巧手类型错误
PARAM_PART_ERROR	3001	部位参数错误
HAND_FAULT_ERROR	4001	设备故障错误
HAND_ENABLE_ERROR	4002	设备使能错误
SEND_FILE_OPERATE_ERROR	5001	文件操作异常
SEND_FRAME_1_ERROR	5002	首帧发送失败
SEND_FRAME_N_ERROR	5003	中间帧发送失败
SEND_FRAME_LAST_ERROR	5004	最后帧发送失败
SEND_EXTRA_ACTION_ERROR	5005	确认动作失败
FILE_SEND_SUCCESS	5006	文件发送成功
UPGRADE_REQUEST_ERROR	5007	请求升级指令失败
UPGRADE_EXECUTE_ERROR	5008	升级指令执行失败
UPGRADE_TYPE_ERROR	5009	升级类型错误

1.67 upgradeSensor

- **功能概述**

升级灵巧手固件（主控板 / 电驱板）。

- **函数原型**

```
1  UpgradeErrorCode upgradeFirmware(int16_t slave_address, const std::string& file_path);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址
file_path	const std::string&	输入	固件文件路径

- **返回值**

返回 UpgradeErrorCode 枚举值, 含义见 1.66 节表格。

1.68 getUpgradeInfo

- **功能概述**

获取固件升级进度信息。如需使用, 请在调用 upgradeFirmware 之前, 另起一个独立线程循环调用本接口轮询进度。

- **函数原型**

```
1  const std::vector<UpgradeProcess> getUpgradeInfo(int16_t slave_address);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
slave_address	int16_t	输入	从站地址

- **返回值**

返回 UpgradeErrorCode 枚举值, 常见取值（同 1.66 节 upgradeFirmware）:

字段	类型	说明
upgrade_type	FirmwareUpgradeType	升级类型（枚举, 见下表）
part	list[int]	升级的固件部位编号
process_data	int	升级进度值, 范围 [0, 100]
status	std::string	升级状态说明文字

- **UpgradeFileAndPart 结构说明**

枚举值	数值	说明
INVALID_UPGRADE	0	无效的升级类型
MAINBOARD_UPGRADE	1	主控板升级
DRIVER_UPGRADE	2	电驱板升级
SENSOR_UPGRADE	3	传感器升级

2. 相机控制类 DexH15Camera

负责灵巧手配套相机设备的连接、图像采集与参数配置。

2.1 connectCameraDevice

- **功能概述**

  打开并连接指定相机设备。

- **函数原型**

```
1  void connectCameraDevice(const std::string& camera_port_num);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
camera_port_num	const std::string&	输入	相机设备编号

- **返回值**

  无返回值（void）。连接失败时抛出异常。

2.2 releaseCameraDevice

- **功能概述**

  释放相机设备资源，断开连接。

- **函数原型**

```
1  void releaseCameraDevice();
```

- **参数说明**

  无

- **返回值**

  无返回值（void）。

2.3 getFrame

- **功能概述**

  从相机采集当前帧图像。

- **函数原型**

```
1  cv::Mat getFrame();
```

- **参数说明**

  无

- **返回值**

  返回 cv::Mat 类型的图像帧，默认分辨率为 640×480，帧率 30 帧。若相机未打开，则返回空 Mat。

2.4 showImage

- **功能概述**

  在窗口中显示指定图像帧。

- **函数原型**

```
1  void showImage(const std::string& imgName, const cv::Mat& img);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
imgName	const std::string&	输入	显示窗口的名称
img	const cv::Mat&	输入	要显示的图像

- **返回值**

  无返回值（void）。

2.5 setCameraConfig

- **功能概述**

  设置相机的分辨率与帧率参数。

- **函数原型**

```
1  bool setCameraConfig(int width, int height, float fps);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
width	int	输入	图像分辨率宽度（像素）
height	int	输入	图像分辨率高度（像素）
fps	float	输入	相机帧率（帧 / 秒）

- **返回值**

  true: 配置成功
  false: 配置失败

- **相机参数说明**

  设置相机参数，包括分辨率和帧率。只支持 640x480、1280x720、1920x1080 分辨率。帧率只支持 640x480、1280x720 分辨率下的 30fps 以及 1920x1080 分辨率下的 60fps

3. 运动学解析类 DexH15Kinematic

负责灵巧手关节角度与电机位置之间的正逆运动学转换，以及归一化角度与真实角度的互相换算。

3.1 getSDKVersion

- **功能概述**

  获取当前 **SDK** 的版本号。

- **函数原型**

```
1  const std::string getSDKVersion();
```

- **参数说明**

  无

- **返回值**

  返回 **SDK** 版本号字符串（std::string）。

3.2 inverseKinematicParse

- **功能概述**

逆运动学解析: 输入 11 个可调关节的真实角度值, 计算并输出对应的 7 个电机位置值。

- **函数原型**

```
int inverseKinematicParse(const HandType hand_type,
                          const std::vector<double>& real_angle,
                          std::vector<int16_t>& motor_pos,
                          const std::string& hardware_version = "");
```

- **参数说明**

参数名	类型	输入 / 输出	说明
hand_type	const HandType	输入	左手或右手类型
real_angle	const std::vector<double>&	输入	11 个可调关节的真实角度值集合 (单位: °)
motor_pos	std::vector<int16_t>&	输出	解析得到的 7 个电机位置信息
hardware_version	const std::string&	输入	硬件版本 (可选, 默认为空)

- **real_angle 元素说明**

顺序依次为: 食指 (2 个) → 中指 (2 个) → 无名指 (2 个) → 小拇指 (2 个) → 大拇指 (3 个)。

索引	关节标识	对应手指	真实角度范围 (°)	说明
0	index_joint1	食指	[0, 80]	近端关节, 弯曲伸展
1	index_joint2	食指	[0, 80]	末端关节, 弯曲伸展
2	middle_joint1	中指	[0, 80]	近端关节, 弯曲伸展
3	middle_joint2	中指	[0, 80]	末端关节, 弯曲伸展
4	ring_joint1	无名指	[0, 80]	近端关节, 弯曲伸展
5	ring_joint2	无名指	[0, 80]	末端关节, 弯曲伸展
6	little_joint1	小拇指	[0, 80]	近端关节, 弯曲伸展
7	little_joint2	小拇指	[0, 80]	末端关节, 弯曲伸展
8	thumb_joint1	大拇指	[0, 109.6]	掌指双轴关节, 侧位摆动
9	thumb_joint2	大拇指	[0, 87]	近端关节, 弯曲伸展
10	thumb_joint3	大拇指	[0, 120.1]	末端关节, 弯曲伸展

- **返回值**

1: 解析成功
0: 解析失败

3.3 forwardKinematicParse

- **功能概述**

正运动学解析: 输入 7 个电机位置值及 8 个磁编数据, 计算并输出关节的真实角度值。当 return_joint_flag 为 false 时输出 11 个可调关节角度; 为 true 时额外输出 4 个末端指骨 (DIP) 关节, 共输出 15 个关节角度。

- **函数原型**

```
int forwardKinematicParse(const HandType hand_type,
                          const std::vector<int16_t>& motor_pos,
                          const std::vector<int16_t>& magnetic_encoder,
                          std::vector<double>& real_angle,
                          const std::string& hardware_version = "");
                          bool return_joint_flag = false);
```

- **参数说明**

参数名	类型	输入 / 输出	说明
hand_type	const HandType	输入	左手或右手类型
motor_pos	const std::vector<int16_t>&	输入	7 个电机位置值集合, 取值范围 [0, 10000]
magnetic_encoder	const std::vector<int16_t>&	输入	8 个磁编码器数据集合
real_angle	std::vector<double>&	输出	解析得到的关节真实角度值 (单位: °), 元素个数由 return_joint_flag 决定
hardware_version	const std::string&	输入	硬件版本 (可选, 默认为空)
return_joint_flag	bool	输入	控制返回关节数量的标志, 默认 false (返回 11 个可调关节, 为 true 时返回 15 个关节, 即 11 个可调关节加 4 个末端指骨 DIP 关节)

- **返回值**

1: 解析成功
0: 解析失败

3.4 calculateRealAngle（运动学类）

- **功能概述**

根据 11 个可调关节的归一化角度集合，计算对应的真实物理角度（单位：度）。

- **函数原型**

```
int calculateRealAngle(const std::vector<double>& normalize_angle,
                       std::vector<double>& real_angle,
                       const std::string& hardware_version = "");
```

- **参数说明**

参数名	类型	输入 / 输出	说明
normalize_angle	const std::vector<double>&	输入	11 个关节的归一化角度集合
real_angle	std::vector<double>&	输出	计算得到的 11 个真实角度集合（单位：°）
hardware_version	const std::string&	输入	硬件版本（可选，默认为空）

- **normalize_angle 元素说明**

顺序依次为：食指（2 个）→ 中指（2 个）→ 无名指（2 个）→ 小拇指（2 个）→ 大拇指（3 个）。

索引	关节标识	对应手指	归一化输入范围	说明
0	index_joint1	食指	[0, 1000]	近端关节，弯曲伸展
1	index_joint2	食指	[0, 1000]	末端关节，弯曲伸展
2	middle_joint1	中指	[0, 1000]	近端关节，弯曲伸展
3	middle_joint2	中指	[0, 1000]	末端关节，弯曲伸展
4	ring_joint1	无名指	[0, 1000]	近端关节，弯曲伸展
5	ring_joint2	无名指	[0, 1000]	末端关节，弯曲伸展
6	little_joint1	小拇指	[0, 1000]	近端关节，弯曲伸展
7	little_joint2	小拇指	[0, 1000]	末端关节，弯曲伸展
8	thumb_joint1	大拇指	[-500, 500]	掌指双轴关节，侧位摆动
9	thumb_joint2	大拇指	[0, 1000]	近端关节，弯曲伸展
10	thumb_joint3	大拇指	[0, 1000]	末端关节，弯曲伸展

- **返回值**

1：计算成功
0：计算失败

3.5 calculateNormalizeAngle（运动学类）

- **功能概述**

根据 11 个可调关节的真实物理角度，计算对应的归一化角度集合。

- **函数原型**

```
int calculateNormalizeAngle(const std::vector<double>& real_angle,
                            std::vector<double>& normalize_angle,
                            const std::string& hardware_version = "");
```

- **参数说明**

参数名	类型	输入 / 输出	说明
real_angle	const std::vector<double>&	输入	11 个关节的真实角度集合（单位：°）
normalize_angle	std::vector<double>&	输出	计算得到的 11 个归一化角度集合
hardware_version	const std::string&	输入	硬件版本（可选，默认为空）

- **real_angle 元素说明**

顺序依次为：食指（2 个）→ 中指（2 个）→ 无名指（2 个）→ 小拇指（2 个）→ 大拇指（3 个）。

索引	关节标识	对应手指	真实角度范围（°）	说明
0	index_joint1	食指	[0, 80]	近端关节，弯曲伸展
1	index_joint2	食指	[0, 80]	末端关节，弯曲伸展
2	middle_joint1	中指	[0, 80]	近端关节，弯曲伸展
3	middle_joint2	中指	[0, 80]	末端关节，弯曲伸展
4	ring_joint1	无名指	[0, 80]	近端关节，弯曲伸展
5	ring_joint2	无名指	[0, 80]	末端关节，弯曲伸展
6	little_joint1	小拇指	[0, 80]	近端关节，弯曲伸展
7	little_joint2	小拇指	[0, 80]	末端关节，弯曲伸展
8	thumb_joint1	大拇指	[0, 109.6]	掌指双轴关节，侧位摆动
9	thumb_joint2	大拇指	[0, 87]	近端关节，弯曲伸展
10	thumb_joint3	大拇指	[0, 120.1]	末端关节，弯曲伸展

- **返回值**

1：计算成功
0：计算失败

PaXini



帕西尼感知科技

电话：**帕西尼·深圳**：0755-23574593

帕西尼·上海：021-54561536

帕西尼·天津：022-82411882

邮箱：**mkt@paxini.com**

官网：**www.paxini.com**

地址：**帕西尼感知科技（深圳）有限公司**：广东省深圳市宝安区海城路 5 号前城中心 34F-35F

帕西尼感知科技研究分部（深圳）：广东省深圳市南山区万科云城六期一栋云中城

帕西尼感知科技智能实验 & 制造中心（深圳）：广东省深圳市宝安区永丰社区

帕西尼感知科技（上海）有限公司：上海市徐汇区桂平路 391 号新漕河泾国际商务中心

帕西尼（天津）Super EID Factory：天津市河西区天津空天信息产业园

帕西尼感知科技制造研发中心（苏州）：江苏省苏州市张家港市经开区华夏科技园

京东 POP 店

天猫旗舰店

京东自营店

关注我们 了解更多